Exercices - Encodage de texte

Compléments python

La fonction `ord` permet d'avoir le point de code d'un caractère, en décimal. On rappelle qu'Unicode est compatible avec l'ascii et l'ascii étendu. De ce fait les codes ascii et les points de code correspondent pour les caractères de code ascii inférieur à 255.

```
ord('y')
```

```
121
```

Une conversion entier $\longrightarrow$ hexadécimal peut être réalisée avec la fonction `hex`.

```
hex(121)
```

```
'0x79'
```

Une chaîne de caractères peut être indexée comme les tableaux. On peut alors y réaliser des itérations.

```
ch1 = "Ma chaîne"
print('longueur: ', len(ch1))
for i in range(len(ch1)):
    print(ch1[i])
```

```
longueur:  9
M
a

c
h
a
î
n
e
```

Comme avec les tableaux, on peut aussi réaliser une itération sur les éléments.

```
for lettre in ch1:
    print(lettre)
```

```
M
a

c
h
a
î
n
e
```

Exercice 1

Donner le codage ASCII des deux chaînes de caractères Python ci-dessous. Attention, on ne code pas les marques de chaîne `'` ou `"` !

```
'bonjour tout le monde!'
```

```
"""programmer en
Python"""
```

Exercice 2

Écrire une fonction `printASCII(s)` qui renvoie un tableau contenant les codes ASCII au format hexadécimal, d'une chaîne de caractères `s` passée en paramètre. Utiliser cette fonction pour vérifier les réponses de l'exercice précédent.
On pourra utiliser la cellule suivante en complétant les zones où `...` apparaissent.

```
def printASCII(s):
    """
    renvoie un tableau contenant le code ascii d'une chaine s, sous forme hexadécimale;
    s: chaine (str)
    """
    code_ascii = [0] * ... # Création d'un tableau de ... éléments
    for i in range(len(s)):
        code_ascii[i] = ...
    return ...
```

Exercice 3

Écrire une fonction `printUTF8(s)` qui prend une chaîne `s` en paramètre et qui renvoie un tableau contenant les octets écrits en binaire résultant de l'encodage de `s` en UTF-8. Tester votre fonction sur le mot `élégance`. Utiliser la fonction `bin_8bits` qui permet de convertir un entier n en binaire sur 8 bits (un octet).
On pourra utiliser la cellule suivante en complétant les zones où `...` apparaissent.

```python
def bin_8bits(n):
    """ Renvoie la représentation binaire de n sur 1 octet. """
    return f"{n:08b}"


def printUTF8(s):
    """
    Renvoie la représentation binaire d'une chaîne s, encodée en utf-8, sous la
    forme d'un tableau d'octets écrits en binaire.
    s: chaine (str)
    """

    octets = s.encode('utf8') # Encodage de s en utf-8 dans un tableau d'octets
    repr_binaire = [0] * ...
    for i in range(...):
        repr_binaire[i] = bin_8bits(...)
    return ...
```

```python
# Cellule de TEST
res = ['11000011',
       '10101001',
       '01101100',
       '11000011',
       '10101001',
       '01100111',
       '01100001',
       '01101110',
       '01100011',
       '01100101'
      ]
assert printUTF8("élégance") == res
print("Validé")
```

Exercice 4**

Ecrire une fonction `est_utf8_valide(n1, n2, n3)` qui renvoie `True` si la suite d'octets (*représentée en base 10*) `n1`, `n2`, `n3` représente une séquence valide UTF-8. Elle renvoie `False` sinon.

Quelques indices

Une suite de 3 octets est un encodage UTF-8 valide si elle possède une des formes suivantes:

1. `1110 xxxx 10xx xxxx 10xx xxxx`: 1 caractère codé sur 3 octets;
2. `110x xxxx 10xx xxxx 0xxx xxxx`: 1 caractère codé sur 2 octets suivi de 1 caractère codé sur 1 octet;
3. `0xxx xxxx 110x xxxx 10xx xxxx`: 1 caractère codé sur 1 octet suivi de 1 caractère codé sur 2 octets;
4. `0xxx xxxx 0xxx xxxx 0xxx xxxx`: 3 caractères codés chacun sur 1 octet.

```python
def est_utf8_valide(n1, n2, n3):
    """
    renvoie True si n1, n2, n3 est une suite valide en UTF-8, False sinon.
    n1, n2, n3: entiers naturels strictement inférieurs à 256.
    """
    # On stoppe la fonction si la précondition n'est pas assurée.
    assert n1 < 256 and n2 < 256 and n3 < 256

    # Cas 3 & 4 --> le 1er octet est compris entre 0 et 0111 1111 soit 127 en décimal
    if n1 < ...:
        if n2 < ... and n3 < 128: # Cas n° 4
            return True
        if (191 < n2 < 224) and (127 < n3 < 192): # Cas n° 3
            return True
    # Cas 2
    if ... < n1 < ...:
        if (127 < n2 < 192) and (n3 < 128):
            return True
    # Cas 1 (forcé)
    if (... < n1) and (127 < n2 < 192 and 127 < n3 < 192):
        return True

    return False
```

```python
# Cellule de TESTS
assert est_utf8_valide(227, 180, 140)
assert est_utf8_valide(126, 64, 100)
assert not est_utf8_valide(198, 129, 129)
print("Validé !")
```